EE-222 Data Structures and Algorithms

BS(CE)-2k21

LAB REPORT#12

Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz

Department of Computer Engineering

HITEC University Taxila

	Presentation (Format(0.5)+ Title (0.5)+ Conclusion(1) +Procedure(2) +Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	**5**	**5**	**5**	**15**
Obtained				

_____

Lab Instructor.
Kaynat Rana

Experiment #12

Implementation of Binary Search Tree

Objective:

The objective of this lab is to understand the basic concept of Trees

- Binary Search Tree
- Insertion in a BST
- Deletion in a BST
- Traversing a Tree

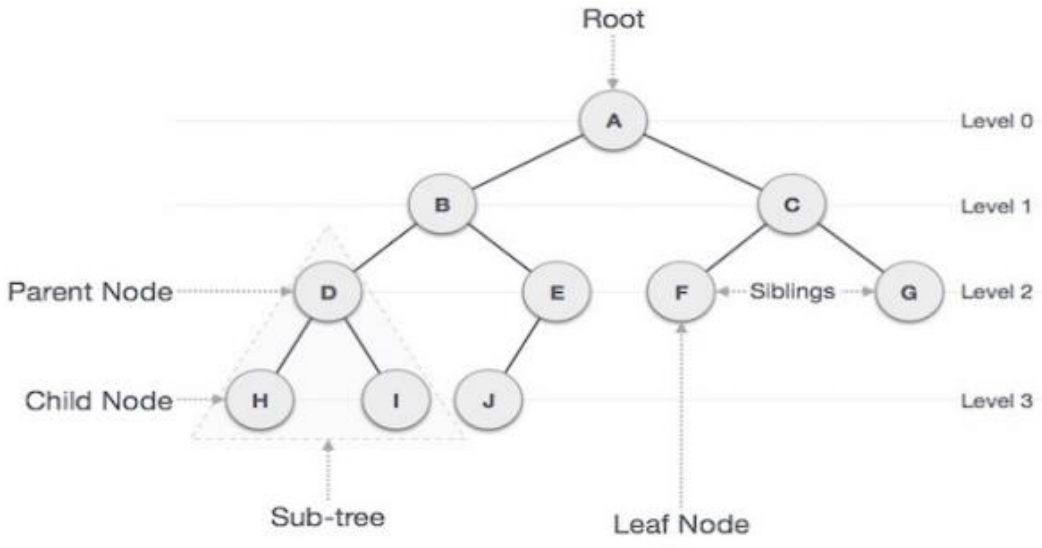
Equipment Used:

Microsoft Visual Studio

Procedure:

Trees:

Tree represents the nodes connected by edges. We will discuss binary tree or binary search tree specifically. Binary Tree is a special data structure used for data storage purposes. A binary tree has a special condition that each node can have a maximum of two children. A binary tree has the benefits of both an ordered array and a linked list as search is as quick as in a sorted array and insertion or deletion operation are as fast as in linked list.



Following are the important terms with respect to tree.

- **Path:**

Path refers to the sequence of nodes along the edges of a tree.

- **Root:**

The node at the top of the tree is called root. There is only one root per tree and one path from the root node to any node.

- **Parent:**

Any node except the root node has one edge upward to a node called parent.

- **Child:**

The node below a given node connected by its edge downward is called its child node.

- **Leaf:**

The node which does not have any child node is called the leaf node.

- **Subtree:**

Subtree represents the descendants of a node.

- **Visiting:**

Visiting refers to checking the value of a node when control is on the node.

- **Traversing:**

Traversing means passing through nodes in a specific order.

- **Levels:**

Level of a node represents the generation of a node. If the root node is at level 0, then its next child node is at level 1, its grandchild is at level 2, and so on.

- **Keys:**

Key represents a value of a node based on which a search operation is to be carried out for a node.

BST Basic Operations:

The basic operations that can be performed on a binary search tree data structure, are the following.

- **Insert:**

Inserts an element in a tree/create a tree.

- **Search:**

Searches an element in a tree.

- **Preorder Traversal:**

Traverses a tree in a pre-order manner.

- **Inorder Traversal:**

Traverses a tree in an in-order manner.

- **Postorder Traversal:**

Traverses a tree in a post-order manner.

Binary Search Tree:

Elements in BST are inserted in sorted form. For any node n in a binary search tree:

- The value at n is greater than the values of any of the nodes in its left subtree.
- The value at n is less than the values of any of the nodes in its right subtree.
- Its left and right subtrees are binary search trees.
- Need to define “greater than” and “less than” for the specific data.

Insert Operation:

The very first insertion creates the tree. Afterwards, whenever an element is to be inserted, first locate its proper location. Start searching from the root node, then if the data is less than the key value, search for the empty location in the left subtree and insert the data. Otherwise, search for the empty location in the right subtree and insert the data.

Node for a Tree Data:

```
struct nodeTree
{
char data;
nodeTree *left;
nodeTree *right;
}
* root = NULL;
```

Insertion a new node:

```
void insertTree(char value)
{
nodeTree *temp = new nodeTree;
nodeTree *cur = new nodeTree;
temp->data = value;
temp->left = NULL;
temp->right = NULL;
if(root == NULL)
{
root = temp;
}
else
{
cur = root;
while(cur != NULL)
{
if(value < cur->data)
{
if(cur->left)
{
cur = cur->left;
}
else
{
cur->left = temp;
break;
}
}
else
{
if(cur->right)
{
cur = cur->right;
}
else
{
cur->right = temp;
break;
```

```
}
}
}
}
}
```

BST Traversal:

- **Inorder:**

The ordering is the left subtree, the current node, the right subtree.

- **Preorder:**

The ordering is the current node, the left subtree, the right subtree.

- **Post order:**

The ordering is the left subtree, the right subtree, the current node.

Procedure:

In order Traversal:

Algorithm In order (tree).

1. Traverse the left subtree, i.e., call in order(left-subtree).
2. Visit the root.
3. Traverse the right subtree, i.e., call in order(right-subtree).

Uses of in order:

In case of binary search trees (BST), in order traversal gives nodes in non-decreasing order. To get nodes of BST in non-increasing order, a variation of in order traversal where Inorder traversals reversed, can be used.

Preorder Traversal:

Algorithm Preorder(tree).

1. Visit the root.
2. Traverse the left subtree, i.e., call Preorder(left-subtree).
3. Traverse the right subtree, i.e., call Preorder(right-subtree).

Uses of Preorder:

Preorder traversal is used to create a copy of the tree. Preorder traversal is also used to get prefix expression on of an expression tree.

Post order Traversal:

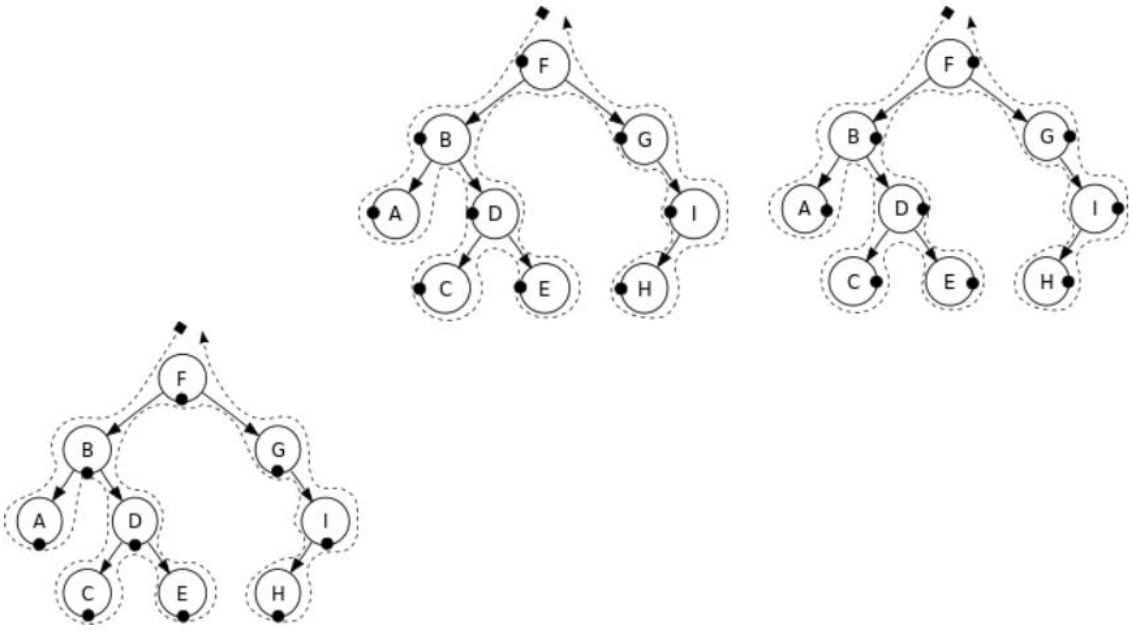
Algorithm Post order (tree).

1. Traverse the left subtree, i.e., call Post order(left-subtree).
2. Traverse the right subtree, i.e., call Post order(right-subtree).
3. Visit the root.

Uses of Post order:

Post order traversal is used to delete the tree. Postorder traversal is also useful to get the postfix expression of an expression tree.

EXAMPLE:



In-order: A, B, C, D, E, F, G, H, I. Pre-order: F, B, A, D, C, E, G, I, H. Post-order: A, C, E, D, B, H, I, G, F

BST Deletion:

While deleting a node in BST, we want to delete the node, but preserve the sub-trees, that the node links to. There are 2 possible situations to be faced when deleting a non-leaf node:

1) The node has one child.
2) The node has two children.

We cannot attach both of the node's subtrees to its parent in case 2.

One solution is to:

a) find a position in the right subtree to attach the left subtree.
b) attach the node's right subtree to the parent.

Tasks

Task 1:

Write a C++ program to create a Binary Search Tree ADT. Your tree must be able to perform the following functions:

a. Insertion in a tree
b. Inorder Traversal
c. Preorder Traversal
d. Postorder Traversal
e. Deletion of any node

CODE:

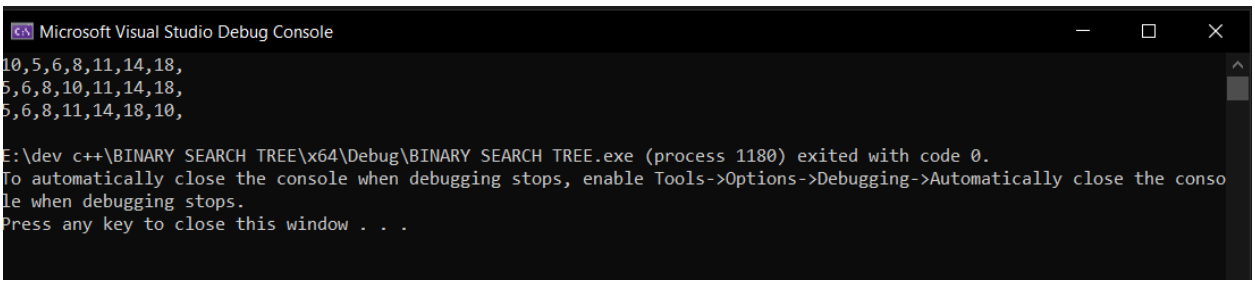
```
#include <iostream>
using namespace std;
struct node
{
	int value;
	node* left;
	node* right;
};
class binaryserachtree
{
public:
	binaryserachtree();
	~binaryserachtree();
	void insert(int key);
	node* search(int key);
	void destroy_tree();
	void inorder_print();
	void postorder_print();
	void preorder_print();
private:
	void destroy_tree(node* leaf);
	void insert(int key, node* leaf);
	node* search(int key, node* leaf);
	void inorder_print(node* leaf);
	void postorder_print(node* leaf);
	void preorder_print(node* leaf);
	node* root;
};
binaryserachtree::binaryserachtree()
{
	root = NULL;
}
binaryserachtree ::~binaryserachtree()
{
	destroy_tree();
}
void binaryserachtree::destroy_tree(node* leaf)
{
	if (leaf != NULL)
	{
		destroy_tree(leaf->left);
		destroy_tree(leaf->right);
		delete leaf;
	}
}
void binaryserachtree::insert(int key, node* leaf)
{
	if (key < leaf->value)
```

```
        {
                if (leaf->left != NULL)
                {
                        insert(key, leaf->left);
                }
                else
                {
                        leaf->left = new node;
                        leaf->left->value = key;
                        leaf->left->left = NULL;
                        leaf->left->right = NULL;
                }
        }
        else if (key >= leaf->value)
        {
                if (leaf->right != NULL)
                {
                        insert(key, leaf->right);
                }
                else
                {
                        leaf->right = new node;
                        leaf->right->value = key;
                        leaf->right->right = NULL;
                        leaf->right->left = NULL;
                }
        }
}
void binaryserachtree::insert(int key)
{
        if (root != NULL)
        {
                insert(key, root);
        }
        else
        {
                root = new node;
                root->value = key;
                root->left = NULL;
                root->right = NULL;
        }
}
node* binaryserachtree::search(int key, node* leaf)
{
        if (leaf != NULL)
        {
                if (key == leaf->value)
                {
                        return leaf;
```

```
                }
                if (key < leaf->value)
{
                        return search(key, leaf->left);
                }
                else
                {
                        return search(key, leaf->right);
                }
        }
        else
        {
                return NULL;
        }
}
node* binaryserachtree::search(int key)
{
        return search(key, root);
}
void binaryserachtree::destroy_tree()
{
        destroy_tree(root);
}
void binaryserachtree::inorder_print()
{
        inorder_print(root);
        cout << "\n";
}
void binaryserachtree::inorder_print(node* leaf)
{
        if (leaf != NULL)
        {
                inorder_print(leaf->left);
                cout << leaf->value << ",";
                inorder_print(leaf->right);
        }
}
void binaryserachtree::postorder_print()
{
        postorder_print(root);
        cout << "\n";
}
void binaryserachtree::postorder_print(node* leaf)
{
        if (leaf != NULL)
        {
                inorder_print(leaf->left);
                inorder_print(leaf->right);
                cout << leaf->value << ",";
```

```
        }
}
void binaryserachtree::preorder_print()
{
        preorder_print(root);
        cout << "\n";
}
void binaryserachtree::preorder_print(node* leaf)
{
        if (leaf != NULL)
        {
                cout << leaf->value << ",";
                inorder_print(leaf->left);
                inorder_print(leaf->right);
        }
}
int main()
{
        //btree tree;
        binaryserachtree* tree = new binaryserachtree();
        tree->insert(10);
        tree->insert(6);
        tree->insert(14);
        tree->insert(5);
        tree->insert(8);
        tree->insert(11);
        tree->insert(18);
        tree->preorder_print();
        tree->inorder_print();
        tree->postorder_print();
        delete tree;
}
```

OUTPUT:



Conclusion:

In conclusion, the implementation of a Binary Search Tree is a useful data structure for efficiently organizing and searching data. It allows for fast insertion, deletion, and searching of elements, with a time complexity of O(log n) in the average case and O(n) in the worst case.